

LONG XUYỀN FOOD

Kiến trúc & công nghệ Production-Ready

Tóm tắt 24 quyết định kiến trúc đã chốt cho Web + Mobile

Mục tiêu: xây dựng nền tảng khám phá, đánh giá và đề xuất địa điểm ăn uống tại Long Xuyên; ưu tiên khả năng mở rộng, phát triển mobile về sau, quản lý nhiều ảnh feedback và triển khai production thực tế.

1. Định vị sản phẩm

Long Xuyên Food không chỉ là website review quán ăn mà là nền tảng khám phá, đánh giá và đề xuất địa điểm ăn uống tại Long Xuyên.

2. Backend

Chọn Laravel 12 + PHP 8.4. Laravel phù hợp với tốc độ phát triển, hệ sinh thái và các yêu cầu API, authentication, queue, cache, notification.

3. Kiến trúc Client-API

Không phụ thuộc vào Blade cho toàn bộ giao diện. Backend cung cấp REST API để Web và Mobile dùng chung business logic.

4. Web Frontend

React + TypeScript. Tách frontend giúp web có kiến trúc tương đồng với mobile và dễ mở rộng.

5. Mobile tương lai

Flutter + Dart. Mobile sử dụng cùng Laravel REST API, không xây backend riêng.

6. API Versioning

Dùng /api/v1 ngay từ đầu. Khi có breaking change có thể phát triển /api/v2 mà không phá client cũ.

7. Authentication

Laravel Sanctum cho SPA/mobile authentication thay vì tự xây cơ chế token riêng.

8. Database chính

PostgreSQL. Phù hợp với quan hệ dữ liệu phức tạp, tìm kiếm, JSON, ranking và khả năng mở rộng.

9. Geospatial

Dùng PostGIS khi cần xử lý vị trí. Cho phép tìm quán trong bán kính, gần người dùng và các truy vấn địa lý hiệu quả.

10. Cache

Redis dùng cho cache như trending restaurants, popular dishes, restaurant detail và dữ liệu truy cập thường xuyên.

11. Queue

Redis + Laravel Horizon cho background jobs: xử lý ảnh, AI moderation, AI summary, notification và các tác vụ nặng.

12. Không lưu ảnh trong DB

PostgreSQL chỉ lưu metadata và storage key; file ảnh nằm trên Object Storage.

13. Image/Object Storage

Chọn Cloudflare R2 làm storage chính cho ảnh restaurant, review, dish và avatar.

14. Lý do chọn R2

R2 tương thích S3 API và không tính phí egress Internet theo mô hình hiện tại, phù hợp với ứng dụng có nhiều ảnh người dùng.

15. CDN/WAF

Dùng Cloudflare phía trước hệ thống cho DNS/CDN/WAF và hỗ trợ phân phối tài nguyên.

16. Image pipeline

Không phục vụ ảnh gốc cho mọi request. Tạo thumbnail, medium và large; ưu tiên WebP/AVIF để giảm kích thước.

17. Image organization

R2 phân chia theo resource: restaurants/{id}, reviews/{id}, avatars/{id}, dishes/{id}; database lưu key tương ứng.

18. Storage abstraction

Business logic không phụ thuộc trực tiếp vào R2. Tạo Media/Storage service hoặc interface để sau này có thể đổi sang S3/Cloudinary.

19. Search

Giai đoạn đầu dùng PostgreSQL Full-Text Search. Khi dữ liệu lớn mới cân nhắc Meilisearch hoặc Typesense.

20. AI architecture

AI được tách thành AI Service/Provider abstraction, phục vụ review summary, recommendation, moderation và semantic search; tránh khóa chặt business logic vào một nhà cung cấp.

21. Recommendation

Bắt đầu bằng rule-based recommendation; sau khi có dữ liệu mới nâng cấp sang recommendation system/AI.

22. Production infrastructure

Docker + CI/CD bằng GitHub Actions; backend có thể triển khai trên AWS, DigitalOcean hoặc Laravel Cloud tùy ngân sách và quy mô.

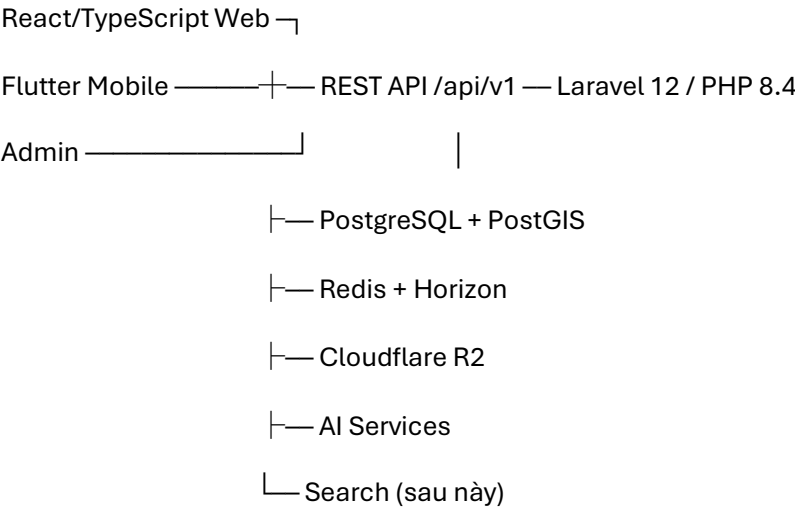
23. Monitoring

Dùng Sentry kết hợp application logs để theo dõi exception, performance và lỗi production.

24. Lộ trình mở rộng

MVP: Laravel + PostgreSQL + React + R2 → Production: Redis + Queue + Horizon + CDN + Sentry + CI/CD → Scale: Search Engine + Recommendation + AI + workers/read replica → Mobile: Flutter dùng cùng API.

Kiến trúc tổng thể đã chốt



Cloudflare: DNS / CDN / WAF

Stack cuối cùng

Layer	Công nghệ	Vai trò
Backend	Laravel 12 + PHP 8.4	REST API, business logic
Web	React + TypeScript	Web client
Mobile	Flutter + Dart	Mobile client tương lai
Database	PostgreSQL + PostGIS	Dữ liệu nghiệp vụ & vị trí
Cache / Queue	Redis + Horizon	Cache, background jobs
Storage	Cloudflare R2	Ảnh & media
CDN / Security	Cloudflare	CDN, DNS, WAF
Search	PostgreSQL → Meilisearch/Typesense	Tìm kiếm
AI	Provider abstraction	Summary, recommendation, moderation
DevOps	Docker + GitHub Actions	Container & CI/CD
Monitoring	Sentry + Logs	Error/performance monitoring

Nguyên tắc kiến trúc

- API-first: Web và Mobile sử dụng cùng backend.
- Database lưu dữ liệu nghiệp vụ; Object Storage lưu file.
- Không over-engineering ở giai đoạn MVP.
- Thiết kế abstraction cho Storage và AI để tránh vendor lock-in.
- Background jobs cho tác vụ nặng như image processing và AI.
- Version API từ đầu để bảo vệ client cũ.
- Chỉ thêm Search Engine, Recommendation và các thành phần scale khi dữ liệu/thực tế yêu cầu.

Bước tiếp theo được đề xuất

Trước khi bắt đầu code, cần thiết kế Database Schema production và API Contract. Các domain cốt lõi nên bao gồm User, Restaurant, Dish, Review, Media, Rating, Favorite, Check-in, Recommendation, Notification và Report.